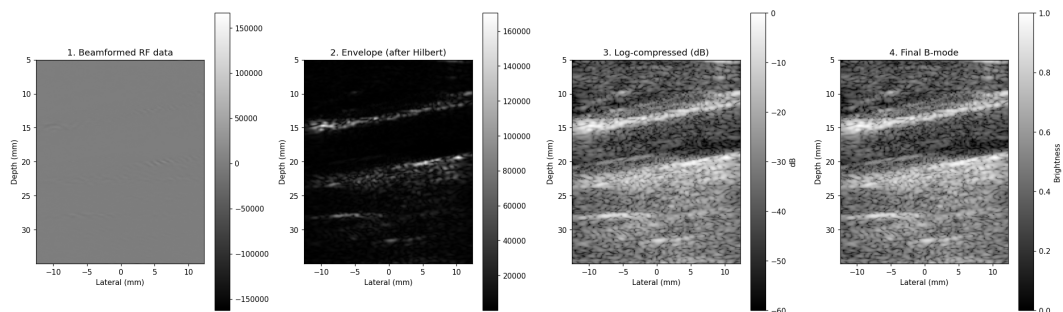


Team 7: B-mode Image Formation

From Team 6's log-compressed echoes to the final grayscale ultrasound image

NITK-USoundSim · 2-D ultrasound simulator · explanation, input, output and run guide

- Stage order: Image Reconstruction (Team 6: Hilbert envelope + log compression) → **B-mode Image Formation (this stage: dynamic range, grayscale brightness, axes, final image)** → Post-Image Processing (Team 8: despeckling).
- Input: Team 6's output files reconstructed_carotid_1/2.npz (log-compressed envelope in dB, 0 dB = peak, 1218 depth samples x 128 scan lines, real in-vivo carotid artery).
- Steps: clip to the dynamic range (60 dB), map to brightness 0..1, depth 5.0-35.0 mm and width -12.5-12.3 mm axes, display, save .npy/.png/_axes.npz for Team 8.
- One command: **python3 run_team7.py** (about 3 seconds). All numbers in this document come from that run.
- Check: this run's image matches the image the notebook saved earlier to 6.49e-06 (max difference).



carotid_1, made by the team notebook in this run: RF and envelope (Team 6's arrays), then this stage's result: dB clipped to 60 dB and the final grayscale B-mode image.

Contents

1 Overview	3
The team's work	3
Files in the folder	3
2 Theory	4
2.1 What arrives from Team 6	4
2.2 Dynamic range and clipping	4
2.3 Mapping dB to brightness and grey levels	4
2.4 Depth axis from the sample time	5
2.5 Lateral axis from the pitch	5
2.6 Display and aspect	5
2.7 Saving for the next stage	6
2.8 What the carotid image shows	6
3 Code explanation	7
3.1 B_mode.ipynb: split between Team 6 and Team 7	7
3.2 run_team7.py (demo runner, written for this folder)	8
3.3 Full listings	8
4 Input	17
5 Output	18
5.1 carotid_1 (60 dB)	18
5.2 carotid_2 (60 dB)	19
5.3 Effect of the dynamic range	20
5.4 Saved files for Team 8	21
5.5 Reproducibility check	22
5.6 Console output	22
6 How to run	26
Troubleshooting	26
7 Check yourself	27

1 Overview

What this stage does and what is in the folder

An ultrasound scanner records echoes. Team 6 (Image Reconstruction) turns the beamformed RF echoes into **echo strength in decibels**: it detects the envelope with the Hilbert transform, normalises it to the strongest echo and applies log compression. Team 7 takes these dB values and forms the picture a doctor looks at: the **B-mode image** (B = brightness). Strong echoes become bright pixels, weak echoes dark pixels.

Place in the pipeline

1. **Image Reconstruction (Team 6)**: Hilbert envelope, normalisation, log compression (dB).
2. **B-mode Image Formation (Team 7, this document)**: dynamic range (clipping), grayscale brightness 0..1, depth and width axes, display and saving of the final grayscale image.
3. **Post-Image Processing (Team 8)**: despeckling of the finished grayscale image.

The team's work

- **B_mode.ipynb**: the team notebook for **real in-vivo carotid artery** data (a blood vessel in the neck). The notebook says the data comes from the EPFL LTS5 public repository (us-non-stationary-deconv): 128-element linear probe, 5 MHz, one plane wave, delay-and-sum beamformed. The notebook contains the whole chain; the sections from '5. APPLY DYNAMIC RANGE' onwards (plus cells 6-9) are this stage's work and are run here.
- **input/reference/bmode_carotid_1.npy**: the image the notebook saved earlier. The runner compares this run's result with it to check that the result is reproducible.

Files in the folder

Path	What it is	Origin
README.md	Short run instructions	written for this demo
requirements.txt	numpy, matplotlib	written for this demo
run_team7.py	Runner: runs this stage's code, saves all results to output/	written for this demo
code/B_mode.ipynb	Team notebook, real carotid data	team original, unchanged
input/reconstructed_carotid_1.npz, _2.npz	Log-compressed envelope and settings	the output of Team 6 (Image Reconstruction), copied here as input
input/reference/bmode_carotid_1.npy, _axes.npz	Image the notebook saved earlier	team original (for the check)
output/	All results of run_team7.py	made by the run
Team_7_..._Explanation.pdf	This document	written for this demo

2 Theory

From dB values to a grayscale picture, in easy words

Envelope detection with the Hilbert transform and log compression are Team 6's work; this stage starts from their result.

2.1 What arrives from Team 6

For every pixel (depth sample z , scan line x) Team 6 delivers the echo strength in decibels relative to the strongest echo of the frame:

$$L(z, x) = 20 \log_{10} \left(\frac{E(z, x)}{\max E} + 10^{-12} \right) \text{ [dB]}, \quad L \leq 0$$

E is Team 6's envelope. 0 dB = the strongest echo; -20 dB = 10 times weaker; -40 dB = 100 times weaker; -60 dB = 1000 times weaker.

In this run L goes from **-90.655 to 0 dB** for `carotid_1` and from **-86.1025 to 0 dB** for `carotid_2` (input file key `envelope_db`, not yet clipped). A screen cannot show 90 dB of detail usefully, and the very weakest values are mostly noise, so this stage chooses which part of the dB range to display.

2.2 Dynamic range and clipping

The **dynamic range** DR is how many dB below the peak are still shown. Every value weaker than -DR is set to -DR (it will be black); nothing is above 0 dB:

$$L_c(z, x) = \text{clip}(L(z, x), -DR, 0)$$

In code (notebook cell 5, section 5): `envelope_db = np.clip(envelope_db, -dynamic_range, 0)`

Dynamic range	carotid_1 black pixels	carotid_2 black pixels	Look of the image
40 dB	26.44 %	30.11 %	high contrast; tissue speckle and blood cut to black
50 dB (project default)	4.90 %	5.74 %	medium
60 dB (notebook setting)	0.50 %	0.63 %	softer and grayer; most weak detail visible

The percentages were measured in this run (Chapter 5.3 shows the images). Most pixels of the carotid lie between about -50 and -20 dB (histogram in Chapter 4), so moving the clip level from -60 to -40 dB turns a quarter of the image black.

2.3 Mapping dB to brightness and grey levels

$$B(z, x) = \frac{L_c(z, x) + DR}{DR} \in [0, 1], \quad G(z, x) = \text{round}(255 B(z, x)) \in \{0, \dots, 255\}$$

-DR dB becomes 0 (black), 0 dB becomes 1 (white); a straight line in between. In code (cell 5, section 6): `bmode = (envelope_db + dynamic_range) / dynamic_range`. The PNG saved by cell 9 stores 0..255 grey levels.

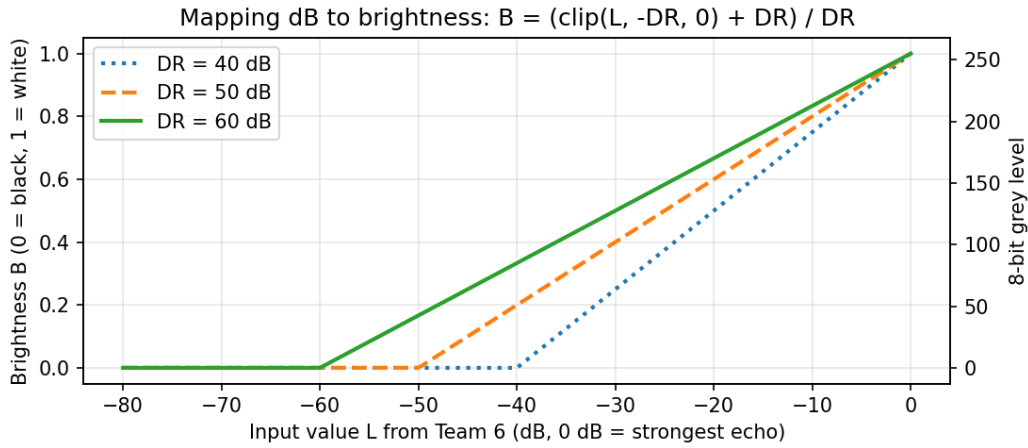


Illustration (made for this document): the gray mapping for three dynamic ranges. A smaller DR gives a steeper line: more contrast, but everything below $-DR$ is black. Right axis: 8-bit grey level.

Example

DR = 60 dB. A pixel at $L = -30$ dB gets $B = (-30 + 60)/60 = 0.5$, grey level 128 (middle grey). A pixel at -75 dB is clipped to -60 dB and gets $B = 0$ (black). The peak at 0 dB gets $B = 1$ (white, 255).

In this run the mean brightness at 60 dB is 0.447 (mean grey level 114.1) for carotid_1 and 0.415 (105.9) for carotid_2.

2.4 Depth axis from the sample time

Row i of the image was recorded at time $t = i / f_s$. The sound goes down to the reflector and back up, so the reflector depth is half the distance travelled. The data starts at 5 mm:

$$z_i = z_0 + \frac{c t_i}{2} = z_0 + \frac{c i}{2 f_s}, \quad f_s = 31.25 \text{ MHz}, \quad c = 1540 \text{ m/s}, \quad z_0 = 5 \text{ mm}$$

One sample is $1540 / (2 \times 31.25 \times 10^6) \text{ m} = 0.0246 \text{ mm}$ deep, so 1218 samples cover **5.0 to 35.0 mm**.

2.5 Lateral axis from the pitch

$$x_k = \left(k - \frac{N}{2}\right) \cdot \text{pitch}, \quad N = 128, \quad \text{pitch} = 0.1953 \text{ mm}$$

This gives **-12.5 to 12.3 mm** (the centre is half a pitch, about 0.1 mm, off zero; not visible).

2.6 Display and aspect

The image is shown with `imshow(..., cmap='gray', vmin=0, vmax=1, extent=[x_first, x_last, z_last, z_first])`. The extent puts real millimetres on both axes and puts depth 5 mm at the top (the probe is at the top). Because the extent is in mm and the default aspect is 'equal', 1 mm deep looks as long as 1 mm wide: the 30 mm x 24.8 mm field of view keeps its true shape, although the array has 1218 rows and only 128 columns. Fixing `vmin=0` and `vmax=1` means brightness 0 is always black and 1 always white.

2.7 Saving for the next stage

- **bmode_<name>.npy**: brightness 0..1 as float32 (the exact values, for processing by Team 8).
- **bmode_<name>.png**: the grayscale picture (plt.imsave, gray colour map, vmin 0, vmax 1).
- **bmode_<name>_axes.npz**: depth_mm and width_mm, so the next stage knows the pixel positions.

2.8 What the carotid image shows

The artery is seen along its length. Its **walls** are strong reflectors and appear as two bright, slanted bands (carotid_1: the upper wall from about 15 mm depth on the left to about 10 mm on the right, the lower wall from about 21 mm to 19 mm). The **blood** reflects little and forms the darker band between them. The grainy texture is **speckle**, which Team 8 reduces afterwards.

3 Code explanation

Which code this stage runs, section by section

3.1 B_mode.ipynb: split between Team 6 and Team 7

The team notebook contains the whole chain from RF to image. The runner splits it at the line # 5. *APPLY DYNAMIC RANGE* (code cell 5, line 16; the split is at the '=' comment line 15 just before it) and runs only this stage's part. The notebook file is not changed.

Notebook code	What it does	Who / run here?
Cell 1	imports; DATA_FILE; np.load of the RF; prints	Team 6. Only the two lines 'import numpy as np' and 'import matplotlib.pyplot as plt' are run
Cell 2	fs, f0, c, pitch, depth_start_mm, dynamic_range = 60	Run (gives dynamic_range; re-assigns the same scalars as in the input file)
Cell 3	Hilbert envelope	Team 6, not run: envelope comes from the input file
Cell 4	Plot of one scan line: RF and envelope	Team 6, not run
Cell 5, lines 1-14	Sections 3-4: normalise, 20·log10	Team 6, not run: envelope_db comes from the input file
Cell 5, lines 15-29	Sections 5-6: clip to [-60, 0]; brightness (dB + 60)/60	Team 7, run
Cell 6	Section 7: depth_mm, width_mm, extent	Team 7, run
Cell 7	Section 8: four panels RF, envelope, clipped dB, B-mode	Team 7, run (RF and envelope panels show Team 6's arrays from the input)
Cell 8	Final B-mode figure	Team 7, run
Cell 9	Section 9: save .npy, .png, _axes.npz	Team 7, run inside output/

Before these pieces run, the runner loads the input file into the namespace (rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitch, depth_start_mm) and sets `DATA_FILE = "carotid_N_rf.npy"`, only because cell 9 derives the output name (bmode_carotid_N) from it.

Section by section (the part run here)

- **Cell 5, section 5:** `np.clip(envelope_db, -dynamic_range, 0)` overwrites `envelope_db` with the clipped values -60..0 dB (equation of 2.2).
- **Cell 5, section 6:** `bmode = (envelope_db + dynamic_range) / dynamic_range`; prints the shape (1218, 128) and the range 0.0 to 1.0 (equation of 2.3).
- **Cell 6:** sample index → time → depth (divide by 2) → mm, plus 5 mm start; width from the pitch; extent = [width first, width last, depth last, depth first] so depth increases downwards.
- **Cell 7:** one figure with four panels and colour bars. The dB panel uses `vmin = -dynamic_range`, `vmax = 0`.
- **Cell 8:** the final image, 7 x 8 inch, gray, `vmin 0`, `vmax 1`, title '`... (60 dB)`'.
- **Cell 9:** name = DATA_FILE without '`_rf.npy`'; saves the three files for Team 8.

3.2 run_team7.py (demo runner, written for this folder)

- Uses matplotlib's file-only mode (Agg) and replaces *plt.show()* by a function that saves each figure in output/.
- Part 1: prints every key of both input files and draws the runner figure *input_envelope_db_overview.png*.
- Part 2: reads *B_mode.ipynb* as JSON, splits cell 5 at the '=' line before '# 5. APPLY DYNAMIC RANGE' and executes the pieces listed in 3.1 in one namespace (cell 9 with working directory output/), for *carotid_1* and *carotid_2*.
- Part 3: runner figure *dynamic_range_40_50_60dB.png* using the notebook's formula on the input *envelope_db*; prints mean brightness and black-pixel percentages; confirms that its 60 dB image is identical to the notebook's *bmode*.
- Part 4: reproducibility check: loads this run's *output/bmode_carotid_1.npy* and the earlier saved *input/reference/bmode_carotid_1.npy*, prints the largest and mean absolute difference, compares the 8-bit grey levels and the axes, and draws *reproducibility_check_60dB.png*.
- Part 5: summary and timings; saves *console_log.txt* and console pictures (its own small function, nothing imported from outside the folder).

3.3 Full listings

B_mode.ipynb - code cell 1 (Team 6's cell; only the two import lines (1-2) are run here)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import hilbert
4
5
6 # =====
7 # 1. LOAD REAL RF DATA
8 # =====
9
10 DATA_FILE = "carotid_1_rf.npy"      # change to "carotid_2_rf.npy" for the second image
11
12 rf = np.load(DATA_FILE)              # shape: (depth samples, scan lines)
13
14 print("RF data shape:", rf.shape)
15 print("Depth samples:", rf.shape[0])
16 print("Scan lines   :", rf.shape[1])
17 print("Minimum:", rf.min())
18 print("Maximum:", rf.max())

```

B_mode.ipynb - code cell 2 (run here)

```

1 # =====
2 # SAMPLING AND PROBE INFORMATION (real probe)
3 # =====
4
5 fs = 31.25e6          # Sampling frequency = 31.25 MHz
6 f0 = 5e6              # Centre frequency of probe = 5 MHz
7 c = 1540              # Speed of sound in tissue = 1540 m/s
8 pitch = 0.1953e-3     # Distance between scan lines = 0.1953 mm
9 depth_start_mm = 5.0  # First sample is at 5 mm depth
10
11 dynamic_range = 60    # 60 dB

```

B_mode.ipynb - code cell 3 (Team 6, not run here)

```

1 # =====
2 # 2. ENVELOPE DETECTION (HILBERT TRANSFORM)
3 # =====
4 # Hilbert transform is applied along depth (axis=0) for every scan line.
5 # abs() of the analytic signal gives the envelope (echo strength).
6

```

```

7 analytic_signal = hilbert(rf, axis=0)
8 envelope = np.abs(analytic_signal)
9
10 print("Envelope shape:", envelope.shape)
11 print("Minimum:", envelope.min())
12 print("Maximum:", envelope.max())

```

B_mode.ipynb - code cell 4 (Team 6, not run here)

```

1 # =====
2 # SHOW ONE SCAN LINE: RF SIGNAL AND ITS ENVELOPE
3 # =====
4
5 line = rf.shape[1] // 2 # middle scan line
6 depth_axis_mm = depth_start_mm + (np.arange(rf.shape[0]) / fs) * c / 2 * 1000
7
8 plt.figure(figsize=(12, 4))
9 plt.plot(depth_axis_mm, rf[:, line], linewidth=0.7, label="RF signal")
10 plt.plot(depth_axis_mm, envelope[:, line], linewidth=1.5, label="Envelope (Hilbert)")
11 plt.xlabel("Depth (mm)")
12 plt.ylabel("Amplitude")
13 plt.title(f"Scan line {line}: RF signal and envelope")
14 plt.legend()
15 plt.show()

```

B_mode.ipynb code cell 5: lines 1-14 are Team 6's (not run here); **lines 15-29 are run here.** Shown as two listings; the second keeps its own numbering from 1 (= cell lines 15-29).

B_mode.ipynb - code cell 5, lines 1-14 (Team 6, not run here)

```

1 # =====
2 # 3. NORMALIZE ENVELOPE
3 # =====
4
5 envelope_norm = envelope / np.max(envelope)
6
7
8 # =====
9 # 4. LOG COMPRESSION (CONVERT TO dB)
10 # =====
11
12 envelope_db = 20 * np.log10(envelope_norm + 1e-12)
13

```

B_mode.ipynb - code cell 5, lines 15-29 (run here)

```

1 # =====
2 # 5. APPLY DYNAMIC RANGE
3 # =====
4
5 envelope_db = np.clip(envelope_db, -dynamic_range, 0)
6
7
8 # =====
9 # 6. CONVERT dB TO BRIGHTNESS (0 to 1)
10 # =====
11
12 bmode = (envelope_db + dynamic_range) / dynamic_range
13
14 print("B-mode shape:", bmode.shape)
15 print("Brightness range:", bmode.min(), "to", bmode.max())

```

B_mode.ipynb - code cell 6 (run here)

```

1 # =====
2 # 7. CALCULATE DEPTH AND WIDTH (mm)
3 # =====
4
5 num_samples, num_scanlines = rf.shape
6
7 sample_indices = np.arange(num_samples)

```

```

8 time = sample_indices / fs                # time of each sample (s)
9 depth = (c * time) / 2                    # divide by 2: sound goes and comes back
10 depth_mm = depth_start_mm + depth * 1000
11
12 width_mm = (np.arange(num_scanlines) - num_scanlines / 2) * pitch * 1000
13
14 print("Depth: %.1f mm to %.1f mm" % (depth_mm[0], depth_mm[-1]))
15 print("Width: %.1f mm to %.1f mm" % (width_mm[0], width_mm[-1]))
16
17 extent = [width_mm[0], width_mm[-1], depth_mm[-1], depth_mm[0]]

```

B_mode.ipynb - code cell 7 (run here)

```

1 # =====
2 # 8. DISPLAY ALL STEPS
3 # =====
4
5 fig, ax = plt.subplots(1, 4, figsize=(20, 6))
6
7 im = ax[0].imshow(rf, cmap="gray", extent=extent)
8 ax[0].set_title("1. Beamformed RF data")
9 plt.colorbar(im, ax=ax[0])
10
11 im = ax[1].imshow(envelope, cmap="gray", extent=extent)
12 ax[1].set_title("2. Envelope (after Hilbert)")
13 plt.colorbar(im, ax=ax[1])
14
15 im = ax[2].imshow(envelope_db, cmap="gray", vmin=-dynamic_range, vmax=0, extent=extent)
16 ax[2].set_title("3. Log-compressed (dB)")
17 plt.colorbar(im, ax=ax[2], label="dB")
18
19 im = ax[3].imshow(bmode, cmap="gray", vmin=0, vmax=1, extent=extent)
20 ax[3].set_title("4. Final B-mode")
21 plt.colorbar(im, ax=ax[3], label="Brightness")
22
23 for a in ax:
24     a.set_xlabel("Lateral (mm)")
25     a.set_ylabel("Depth (mm)")
26
27 plt.tight_layout()
28 plt.show()

```

B_mode.ipynb - code cell 8 (run here)

```

1 # =====
2 # FINAL B-MODE IMAGE
3 # =====
4
5 plt.figure(figsize=(7, 8))
6
7 plt.imshow(bmode, cmap="gray", vmin=0, vmax=1, extent=extent)
8
9 plt.xlabel("Lateral (mm)")
10 plt.ylabel("Depth (mm)")
11 plt.title("Final B-mode Ultrasound Image\nReal in-vivo carotid artery (60 dB)")
12 plt.colorbar(label="Brightness")
13
14 plt.show()

```

B_mode.ipynb - code cell 9 (run here (inside output/))

```

1 # =====
2 # 9. SAVE OUTPUT FOR IMAGE RECONSTRUCTION / POST-PROCESSING TEAM
3 # =====
4
5 name = DATA_FILE.replace("_rf.npy", "")
6
7 np.save(f"bmode_{name}.npy", bmode.astype(np.float32)) # float values 0 to 1
8 plt.imsave(f"bmode_{name}.png", bmode, cmap="gray", vmin=0, vmax=1) # grayscale image
9 np.savez(f"bmode_{name}_axes.npz", depth_mm=depth_mm, width_mm=width_mm)
10
11 print("Saved:", f"bmode_{name}.npy", "|", f"bmode_{name}.png", "|", f"bmode_{name}_axes.npz")
12 print("Image size:", bmode.shape, "(depth samples x scan lines)")

```

run_team7.py (runner)

```

1  """
2  run_team7.py - Team 7: B-mode Image Formation (demo runner)
3
4  One command runs this stage and saves every result in output/:
5
6      python3 run_team7.py      (macOS / Linux)
7      python  run_team7.py      (Windows)
8
9  Place in the simulator
10   Team 6 (Image Reconstruction): Hilbert envelope, normalisation, log compression (dB)
11   -> Team 7 (THIS STAGE): dynamic range (clipping), grayscale brightness 0..1,
12       depth/width axes, display and saving of the final grayscale B-mode image
13   -> Team 8 (Post-Image Processing): despeckling of the finished image
14
15  Input
16   input/reconstructed_carotid_1.npz, input/reconstructed_carotid_2.npz
17   = the output of Team 6 (Image Reconstruction), copied here as input.
18   Keys: rf, envelope, envelope_norm, envelope_db (dB, 0 dB = peak, not yet clipped),
19       fs, f0, c, pitch, depth_start_mm.
20
21  What it does
22   Part 1 Shows the input (Team 6's log-compressed envelope in dB).
23   Part 2 Runs the B-mode part of code/B_mode.ipynb on carotid_1 and carotid_2:
24       code cell 2 (settings, gives dynamic_range), code cell 5 from section
25       "5. APPLY DYNAMIC RANGE" to its end (sections 5 and 6), code cells 6, 7, 8, 9.
26       Code cells 1, 3, 4 and the first half of cell 5 (sections 1 to 4: load RF,
27       Hilbert envelope, normalise, log compression) are Team 6's work; their
28       results come from the input file instead.
29   Part 3 Dynamic-range comparison (40 / 50 / 60 dB) made from the input envelope_db.
30   Part 4 Reproducibility check: this run's output/bmode_carotid_1.npy against the image
31       the notebook saved earlier (input/reference/bmode_carotid_1.npy).
32   Part 5 Summary, timings, console log and console pictures.
33
34  code/B_mode.ipynb is the team's original notebook and is NOT changed.
35  This runner only reads it.
36  """
37
38  import io
39  import json
40  import os
41  import sys
42  import time
43  from pathlib import Path
44
45  sys.dont_write_bytecode = True # keep code/ clean (no __pycache__)
46
47  import matplotlib # noqa: E402
48
49  matplotlib.use("Agg") # save figures to files, no windows
50  import matplotlib.pyplot as plt # noqa: E402
51  import numpy as np # noqa: E402
52
53  ROOT = Path(__file__).resolve().parent
54  CODE = ROOT / "code"
55  INPUT = ROOT / "input"
56  OUTPUT = ROOT / "output"
57  OUTPUT.mkdir(exist_ok=True)
58
59
60  # -----
61  # Console: print to the screen AND keep a copy for output/console_log.txt
62  # -----
63  class Tee(io.TextIOBase):
64      def __init__(self, stream):
65          self.stream = stream
66          self.buffer_text = io.StringIO()
67
68      def write(self, s):
69          self.stream.write(s)
70          self.buffer_text.write(s)
71          return len(s)
72
73      def flush(self):
74          self.stream.flush()

```

```

75
76
77 tee = Tee(sys.stdout)
78 sys.stdout = tee
79
80
81 def banner(text):
82     print()
83     print("=" * 78)
84     print(text)
85     print("=" * 78)
86
87
88 def rng(a):
89     return f"{float(np.min(a)):.6g} to {float(np.max(a)):.6g}"
90
91
92 # -----
93 # plt.show() replacement: save each figure to output/ with a numbered name
94 # -----
95 saved_figures = []
96 _fig_state = {"prefix": "fig", "names": [], "count": 0}
97
98
99 def _save_and_close(*args, **kwargs):
100     _fig_state["count"] += 1
101     n = _fig_state["count"]
102     names = _fig_state["names"]
103     label = names[n - 1] if n <= len(names) else f"figure{n}"
104     path = OUTPUT / f"{_fig_state['prefix']}_{n:02d}_{label}.png"
105     plt.gcf().savefig(path, dpi=110, bbox_inches="tight")
106     plt.close("all")
107     saved_figures.append(path.name)
108     print(f"    [figure saved] output/{path.name}")
109
110
111 plt.show = _save_and_close
112
113
114 def code_cells(notebook_path):
115     """Return the source text of every code cell of a Jupyter notebook."""
116     nb = json.loads(Path(notebook_path).read_text(encoding="utf-8"))
117     return ["".join(c["source"]) for c in nb["cells"] if c["cell_type"] == "code"]
118
119
120 def run_source(src, label, namespace, workdir=None):
121     """Execute one piece of notebook code in the shared namespace."""
122     print(f"--- {label}")
123     start_dir = os.getcwd()
124     try:
125         if workdir is not None:
126             os.chdir(workdir)
127             exec(compile(src, label, "exec"), namespace)
128     finally:
129         os.chdir(start_dir)
130
131
132 # -----
133 # Split code/B_mode.ipynb into Team 6's part and this stage's part
134 # -----
135 NB = CODE / "B_mode.ipynb"
136 cells = code_cells(NB)
137 MARK = "# 5. APPLY DYNAMIC RANGE"
138 cell5 = cells[4]
139 cut = cell5.rindex("# ===", 0, cell5.index(MARK)) # the '=' line just before the marker
140 CELL5_TEAM7 = cell5[cut:] # sections 5 and 6 of code cell 5
141 # cell 1's import lines only (NOT its np.load): numpy and matplotlib.pyplot
142 IMPORTS = "\n".join(ln for ln in cell5[0].splitlines()
143                     if ln.startswith("import numpy") or ln.startswith("import matplotlib.pyplot"))
144
145 HANDOFF_KEYS = ["rf", "envelope", "envelope_norm", "envelope_db", "fs", "f0", "c", "pitch", "depth",
146                "_start_mm"]
147
148
149 def load_handoff(n):
150     """Load Team 6's handoff file into a fresh namespace (arrays + Python scalars)."""
151     d = np.load(INPUT / f"reconstructed_carotid_{n}.npz")

```

```

152     ns = {"__name__": "__main__"}
153     for k in HANDOFF_KEYS:
154         v = d[k]
155         ns[k] = v.item() if v.ndim == 0 else v
156     ns["DATA_FILE"] = f"carotid_{n}_rf.npy" # cell 9 derives the output name from it
157     return ns
158
159
160 timings = {}
161 t_total = time.perf_counter()
162
163 banner("TEAM 7 - B-MODE IMAGE FORMATION (NITK-UsoundSim)")
164 print("Stage order: Team 6 Image Reconstruction (Hilbert envelope + log compression)")
165 print("    -> Team 7 B-mode Image Formation (THIS STAGE: dynamic range, gray levels, axes, f
166       ↪ inal image)")
167 print("    -> Team 8 Post-Image Processing (despeckling)")
168 print("Folder :", ROOT.name)
169 print("Python :", sys.version.split()[0], "| numpy", np.__version__, "| matplotlib", matplotlib.__
170       ↪ version__)
171
172 # -----
173 # PART 1: the input (Team 6's handoff)
174 # -----
175 banner("PART 1: INPUT = output of Team 6 (Image Reconstruction), copied into input/")
176 t = time.perf_counter()
177 handoff = {}
178 for n in (1, 2):
179     d = np.load(INPUT / f"reconstructed_carotid_{n}.npz")
180     handoff[n] = d
181     print(f"input/reconstructed_carotid_{n}.npz")
182     for k in HANDOFF_KEYS:
183         v = d[k]
184         if v.ndim == 0:
185             print(f" {k:<15s} scalar {v.dtype} = {v.item()}")
186         else:
187             print(f" {k:<15s} shape {v.shape}, {v.dtype}, values {rng(v)}")
188
189 fig, ax = plt.subplots(1, 3, figsize=(16, 6), gridspec_kw={"width_ratios": [1, 1, 1.25]})
190 for k, n in enumerate((1, 2)):
191     d = handoff[n]
192     L = d["envelope_db"]
193     nz, nx = L.shape
194     dmm = float(d["depth_start_mm"]) + np.arange(nz) / float(d["fs"]) * float(d["c"]) / 2 * 1000
195     wmm = (np.arange(nx) - nx / 2) * float(d["pitch"]) * 1000
196     im = ax[k].imshow(L, cmap="gray", vmin=float(L.min()), vmax=0, extent=[wmm[0], wmm[-1], dmm[-1]
197     ↪ ], dmm[0])
198     ax[k].set_title(f"reconstructed_carotid_{n}.npz: envelope_db\n(full range {L.min():.1f} to 0 d
199     ↪ B, not clipped)")
200     ax[k].set_xlabel("Lateral (mm)")
201     ax[k].set_ylabel("Depth (mm)")
202     plt.colorbar(im, ax=ax[k], label="dB (0 dB = peak)")
203     ax[2].hist(L.ravel(), bins=180, histtype="step", lw=1.2, label=f"carotid_{n}")
204     for dr, ls in [(40, "-"), (50, "--"), (60, "-.")]:
205         ax[2].axvline(-dr, color="k", ls=ls, lw=1, label=f"-{dr} dB (clip level for DR = {dr} dB)")
206     ax[2].set_xlabel("envelope_db value (dB)")
207     ax[2].set_ylabel("Number of pixels")
208     ax[2].set_title("Histogram of the input dB values")
209     ax[2].legend(fontsize=8)
210     fig.tight_layout()
211     fig.savefig(OUTPUT / "input_envelope_db_overview.png", dpi=110)
212     plt.close(fig)
213     saved_figures.append("input_envelope_db_overview.png")
214     print("    [figure saved] output/input_envelope_db_overview.png (runner figure)")
215     timings["Part 1 input overview"] = time.perf_counter() - t
216
217 # -----
218 # PART 2: this stage's notebook code on the handoff data
219 # -----
220 carotid_figs = ["all_steps", "final_bmode"]
221 ns_all = {}
222 for n in (1, 2):
223     banner(f"PART 2.{n}: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_{n}.npz")
224     print("Loaded into the namespace from the input file: " + ", ".join(HANDOFF_KEYS))
225     print(f"Runner sets DATA_FILE = \"carotid_{n}_rf.npy\" (only used by cell 9 to name the saved fi
226     ↪ les).")
227     print("Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,")
228     print("cell 5 sections 3-4 (normalise, log compression).")

```

```

229     t = time.perf_counter()
230     _fig_state.update(prefix=f"carotid_{n}", names=carotid_figs, count=0)
231     ns = load_handoff(n)
232     run_source(IMPORTS, "cell 1, import lines only: " + IMPORTS.replace("\n", "; "), ns)
233     run_source(cells[1], "B_mode.ipynb code cell 2 (settings; gives dynamic_range)", ns)
234     run_source(CELL5_TEAM7, "B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, bright
235     ↪ ness 0..1)", ns)
236     run_source(cells[5], "B_mode.ipynb code cell 6 (depth and width axes)", ns)
237     run_source(cells[6], "B_mode.ipynb code cell 7 (display all steps)", ns)
238     run_source(cells[7], "B_mode.ipynb code cell 8 (final B-mode image)", ns)
239     run_source(cells[8], "B_mode.ipynb code cell 9 (save, run inside output/)", ns, workdir=OUTPUT
240     ↪ )
241     ns_all[n] = ns
242     timings[f"Part 2.{n} notebook stage, carotid_{n}"] = time.perf_counter() - t
243
244     # -----
245     # PART 3: dynamic range comparison (runner figure, the notebook's formula)
246     # -----
247     banner("PART 3: effect of the dynamic range (runner figure, notebook formula)")
248     print("For DR = 40, 50, 60 dB: B = (clip(envelope_db, -DR, 0) + DR) / DR from the input envelope
249     ↪ _db")
250     t = time.perf_counter()
251     dr_images = {}
252     black = {}
253     for n in (1, 2):
254         L = handoff[n]["envelope_db"]
255         for dr in (40, 50, 60):
256             b = (np.clip(L, -dr, 0) + dr) / dr
257             dr_images[(n, dr)] = b
258             black[(n, dr)] = 100.0 * np.mean(b == 0)
259             print(f"  carotid_{n}, DR {dr} dB: mean brightness {b.mean():.3f}, "
260                   f"black (clipped) pixels {black[(n, dr)]:5.2f} %, pixels at full white {np.sum(b ==
261     ↪ 1)}")
262     ext = ns_all[1]["extent"]
263     fig, ax = plt.subplots(2, 3, figsize=(12, 10.5))
264     for r, n in enumerate((1, 2)):
265         for k, dr in enumerate((40, 50, 60)):
266             im = ax[r, k].imshow(dr_images[(n, dr)], cmap="gray", vmin=0, vmax=1, extent=ext)
267             ax[r, k].set_title(f"carotid_{n}, DR {dr} dB\n(black pixels {black[(n, dr)]:.1f} %)")
268             ax[r, k].set_xlabel("Lateral (mm)")
269             ax[r, k].set_ylabel("Depth (mm)")
270     fig.colorbar(im, ax=ax, fraction=0.025, label="Brightness (0 = black, 1 = white)")
271     fig.suptitle("Runner figure: dynamic range 40 / 50 / 60 dB applied to Team 6's envelope_db")
272     fig.savefig(OUTPUT / "dynamic_range_40_50_60dB.png", dpi=110, bbox_inches="tight")
273     plt.close(fig)
274     saved_figures.append("dynamic_range_40_50_60dB.png")
275     print(f" [figure saved] output/dynamic_range_40_50_60dB.png (runner figure)")
276     same60 = np.array_equal(dr_images[(1, 60)], ns_all[1]["bmode"])
277     print(f" 60 dB image here identical to the notebook's bmode for carotid_1: {same60}")
278     timings["Part 3 dynamic range comparison"] = time.perf_counter() - t
279
280     # -----
281     # PART 4: reproducibility check against the notebook's earlier saved image
282     # -----
283     banner("PART 4: reproducibility check (this run vs the image the notebook saved earlier)")
284     t = time.perf_counter()
285     ref = np.load(INPUT / "reference" / "bmode_carotid_1.npy")
286     ref_axes = np.load(INPUT / "reference" / "bmode_carotid_1_axes.npz")
287     saved_today = np.load(OUTPUT / "bmode_carotid_1.npy")
288     axes_today = np.load(OUTPUT / "bmode_carotid_1_axes.npz")
289     diff = np.abs(saved_today.astype(np.float64) - ref.astype(np.float64))
290     d_nb = float(diff.max())
291     axes_same = all(np.array_equal(ref_axes[k], axes_today[k]) for k in ("depth_mm", "width_mm"))
292     gray_diff = np.abs(np.round(255 * saved_today.astype(np.float64)) - np.round(255 * ref.astype(np.f
293     ↪ loat64)))
294     print(f" this run output/bmode_carotid_1.npy : shape {saved_today.shape}, {saved_today.
295     ↪ dtype}, values {rng(saved_today)}")
296     print(f" reference input/reference/bmode_carotid_1.npy : shape {ref.shape}, {ref.dtype}, values {
297     ↪ rng(ref)}")
298     print(f" max |this run - reference| = {d_nb:.3g}")
299     print(f" mean |this run - reference| = {float(diff.mean()):.3g}")
300     print(f" 8-bit gray levels round(255 * B): {int(np.sum(gray_diff > 0))} of {gray_diff.size} pixel
301     ↪ s differ, "
302           f"by at most {int(gray_diff.max())} level (values that sit right on a rounding boundary)")
303     print(f" depth_mm and width_mm identical to reference : {axes_same}")
304     ok = d_nb < 1e-5 and axes_same
305     print(f" RESULT: ", "PASS - same image (difference is float32 rounding, far below one gray level 1/

```

```

306     ↳ 255 = 0.0039)"
307     if ok else "FAIL - results differ")
308 timings["Part 4 reproducibility check"] = time.perf_counter() - t
309
310 fig, ax = plt.subplots(1, 3, figsize=(13, 6))
311 for a, img, title in [(ax[0], ref, "Saved reference\n(input/reference/bmode_carotid_1.npy)"),
312                      (ax[1], saved_today, "This run\n(output/bmode_carotid_1.npy, 60 dB)"]):
313     a.imshow(img, cmap="gray", vmin=0, vmax=1, extent=ext)
314     a.set_title(title)
315     a.set_xlabel("Lateral (mm)")
316     a.set_ylabel("Depth (mm)")
317 im = ax[2].imshow(diff, cmap="magma", extent=ext)
318 ax[2].set_title(f"|this run - reference|\n(max {d_nb:.2g})")
319 ax[2].set_xlabel("Lateral (mm)")
320 plt.colorbar(im, ax=ax[2])
321 fig.tight_layout()
322 fig.savefig(OUTPUT / "reproducibility_check_60dB.png", dpi=110)
323 plt.close(fig)
324 saved_figures.append("reproducibility_check_60dB.png")
325 print("    [figure saved] output/reproducibility_check_60dB.png (runner figure)")
326
327 # -----
328 # PART 6: summary
329 # -----
330 banner("PART 5: SUMMARY")
331 for n in (1, 2):
332     ns = ns_all[n]
333     L_in = handoff[n]["envelope_db"]
334     print(f"carotid_{n}:")
335     print(f"  input envelope_db   {L_in.shape}, {L_in.dtype}, values {rng(L_in)} dB")
336     print(f"  dynamic range           {ns['dynamic_range']} dB -> clipped dB values {rng(ns['envelope_
337 ↳ db'])} dB")
338     print(f"  B-mode brightness   {ns['bmode'].dtype}, values {rng(ns['bmode'])}, mean {ns['bmode']
339 ↳ .mean():.3f}")
340     print(f"  8-bit gray levels   0 to 255 (round(255 * B)): mean {np.round(255 * ns['bmode']).mean
341 ↳ ():.1f}")
342     print(f"  axes                depth {ns['depth_mm'][0]:.1f} to {ns['depth_mm'][-1]:.1f} mm, "
343           f"lateral {ns['width_mm'][0]:.1f} to {ns['width_mm'][-1]:.1f} mm")
344     print("  black pixels      " + ", ".join(f"{dr} dB: {black[(n, dr)]:.2f} %" for dr in (40, 50
345 ↳ , 60)))
346 print(f"Reproducibility (carotid_1): max |this run - reference| = {d_nb:.3g} ({'PASS' if ok else
347 ↳ 'FAIL'})")
348 print("\nTimings:")
349 for k, v in timings.items():
350     print(f"  {k:<40s} {v:6.2f} s")
351 total = time.perf_counter() - t_total
352 print(f"  {'TOTAL':<40s} {total:6.2f} s")
353
354 print("\nFiles written to output:")
355 for f in sorted(p.name for p in OUTPUT.iterdir() if not p.name.startswith("console_")):
356     print("    ", f)
357 print("    console_log.txt")
358 print("    console_screenshot_1.png, _2.png, ... (pictures of this console text, 75 lines each)")
359 print("\nDone.")
360
361 # -----
362 # Save the console text and a screenshot-style picture of it
363 # -----
364 sys.stdout = tee.stream
365 text = tee.buffer_text.getvalue()
366 (OUTPUT / "console_log.txt").write_text(text, encoding="utf-8")
367
368 lines = []
369 for line in text.splitlines():
370     while len(line) > 125:
371         lines.append(line[:125])
372         line = "    " + line[125:]
373     lines.append(line)
374 # one picture per 75 lines, so each picture fits on a page
375 chunks = [lines[i:i + 75] for i in range(0, len(lines), 75)]
376 shots = []
377 for k, chunk in enumerate(chunks, start=1):
378     h = 0.6 + 0.165 * len(chunk)
379     fig = plt.figure(figsize=(13.5, h), facecolor="#1e1e1e")
380     fig.text(0.01, 1 - 0.25 / h, f"Terminal - python3 run_team7.py (part {k} of {len(chunks)})",
381 ↳ color="#9cdcfe",
382           family="monospace", fontsize=9, va="top", weight="bold")

```

```
383     fig.text(0.01, 1 - 0.55 / h, "\n".join(chunk), color="#e8e8e8", family="monospace",
384             fontsize=8.2, va="top", linespacing=1.25)
385     name = f"console_screenshot_{k}.png"
386     fig.savefig(OUTPUT / name, dpi=100, facecolor="#1e1e1e")
387     plt.close(fig)
388     shots.append(name)
389     print("Console log saved to output/console_log.txt and output/" + ", output/".join(shots))
```

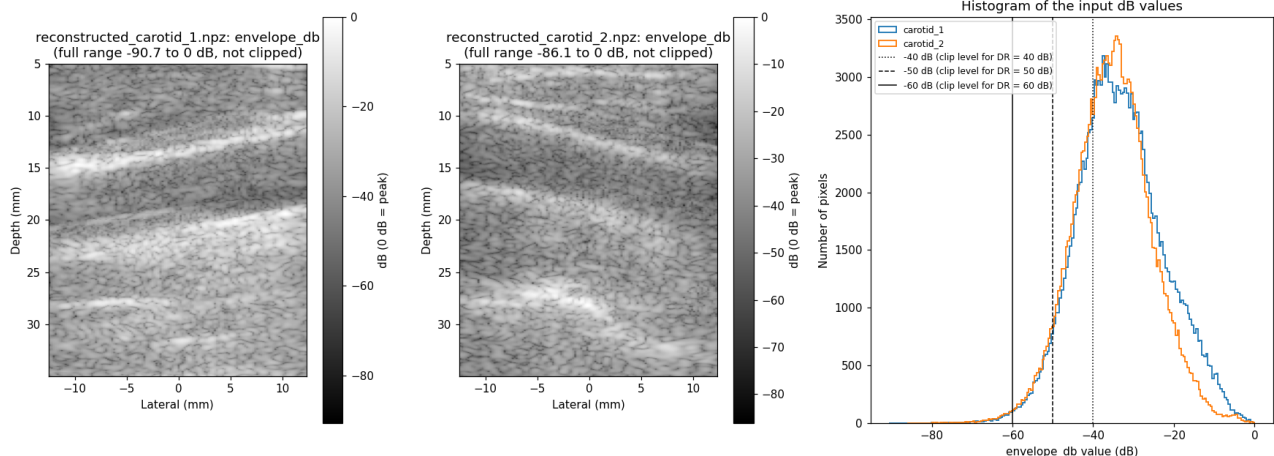
4 Input

Team 6's handoff files

The input files **input/reconstructed_carotid_1.npz** and **input/reconstructed_carotid_2.npz** are the output of Team 6 (Image Reconstruction), copied here as input. They were written with `np.savez` and hold exactly the arrays of the team notebook after its sections 1-4 (load RF, Hilbert envelope, normalise, log compression). Values from this run:

Key	Shape, dtype	carotid_1	carotid_2
rf	(1218, 128) float32	-162577 to 166418	-130418 to 139393
envelope	(1218, 128) float32	4.99583 to 170356	7.11421 to 143632
envelope_norm	(1218, 128) float32	2.93259e-05 to 1	4.95308e-05 to 1
envelope_db	(1218, 128) float32	-90.655 to 0 dB	-86.1025 to 0 dB
fs	scalar float64	31250000.0 (31.25 MHz)	same
f0	scalar float64	5000000.0 (5 MHz)	same
c	scalar float64	1540.0 (m/s)	same
pitch	scalar float64	0.0001953 (m)	same
depth_start_mm	scalar float64	5.0	same

envelope_db is the key this stage works on: dB relative to the frame peak (0 dB = peak), not yet clipped. rf and envelope are used only for the first two panels of the notebook's all-steps figure.



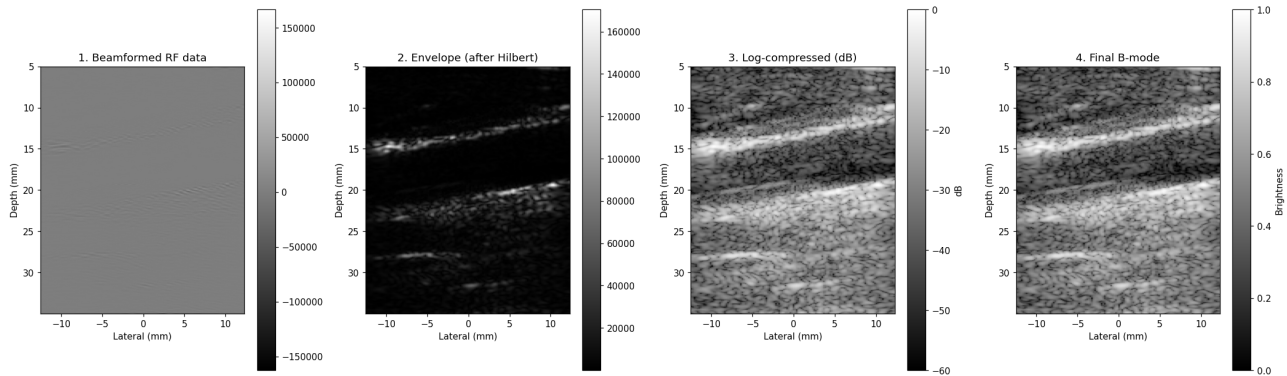
Runner figure `input_envelope_db_overview.png`: `envelope_db` of both frames on their full dB range, and a histogram of all pixel values with the clip levels -40, -50 and -60 dB. Pixels left of a line become black for that dynamic range.

Reference file: `input/reference/bmode_carotid_1.npy (+_axes.npz)` is the image the team notebook saved earlier; it is used only for the reproducibility check.

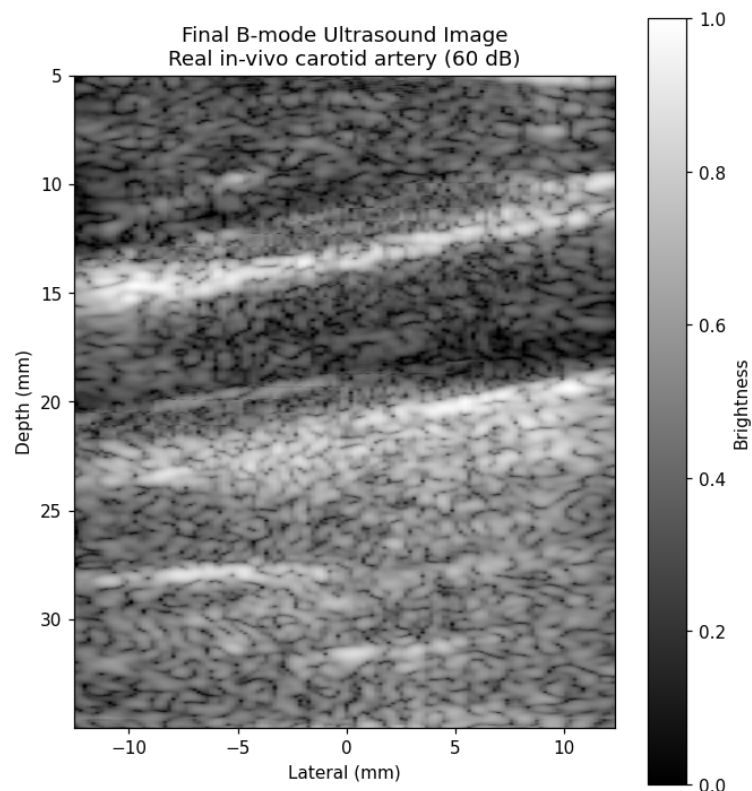
5 Output

Every figure, the saved images and the real console output

5.1 carotid_1 (60 dB)

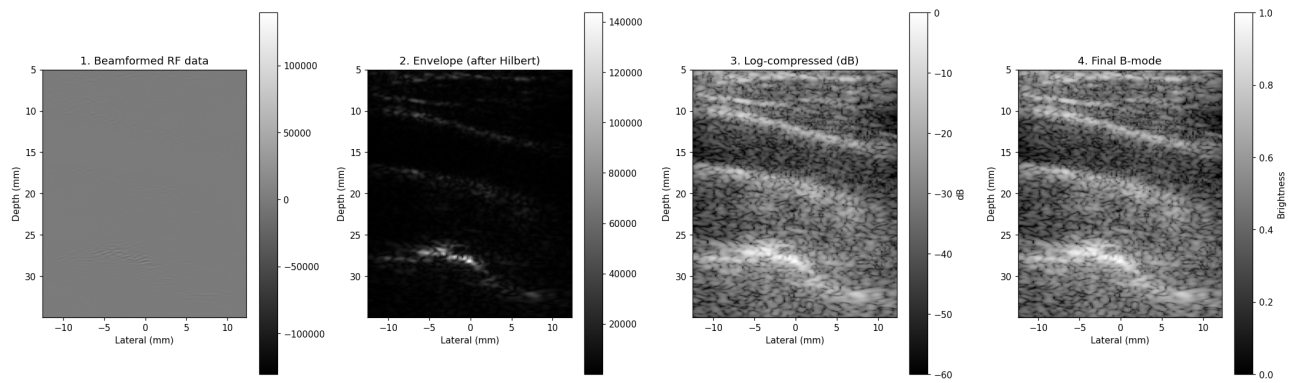


carotid_1_01_all_steps.png (notebook cell 7): (1) the RF and (2) the envelope, both from Team 6's file; (3) the dB values after clipping to -60..0 dB; (4) the brightness 0..1. Panels 3 and 4 look the same because the gray mapping is a straight line; they differ only in the colour-bar units.

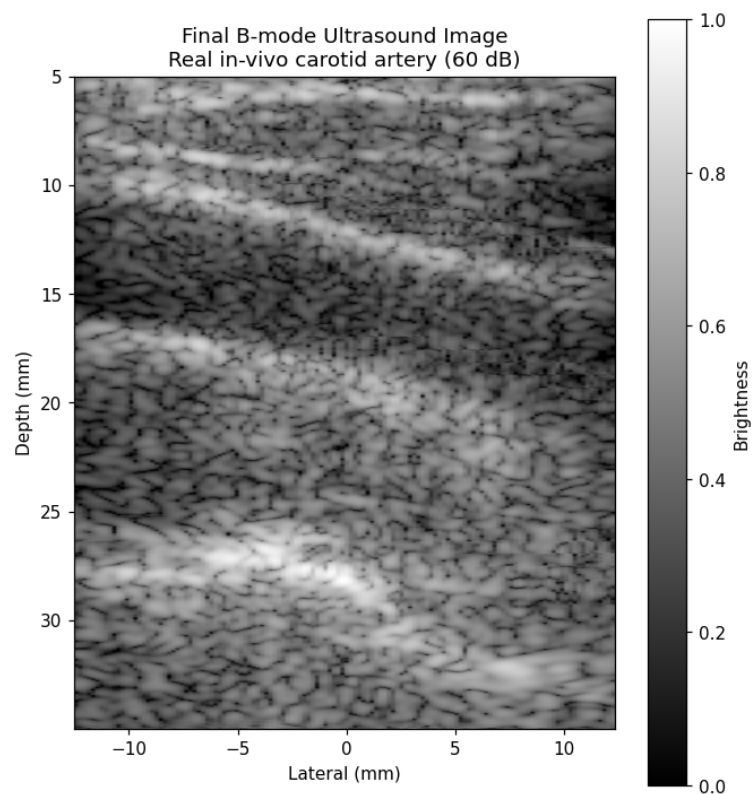


carotid_1_02_final_bmode.png (cell 8): the final B-mode image. Bright slanted bands: vessel walls; darker band between them: blood in the lumen; grainy texture: speckle.

5.2 carotid_2 (60 dB)



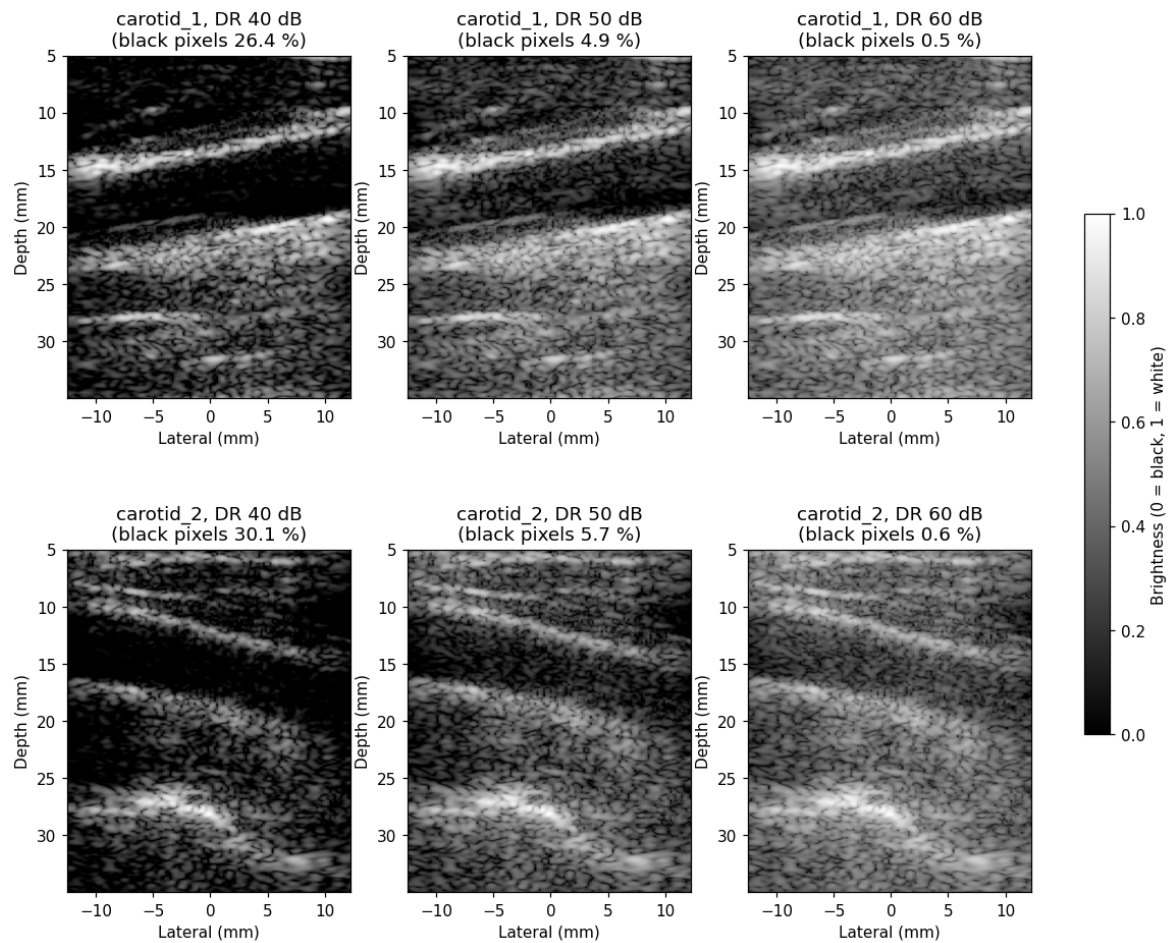
carotid_2_01_all_steps.png: the same four panels for the second frame.



carotid_2_02_final_bmode.png: final B-mode image of the second frame.

5.3 Effect of the dynamic range

Runner figure: dynamic range 40 / 50 / 60 dB applied to Team 6's envelope_db

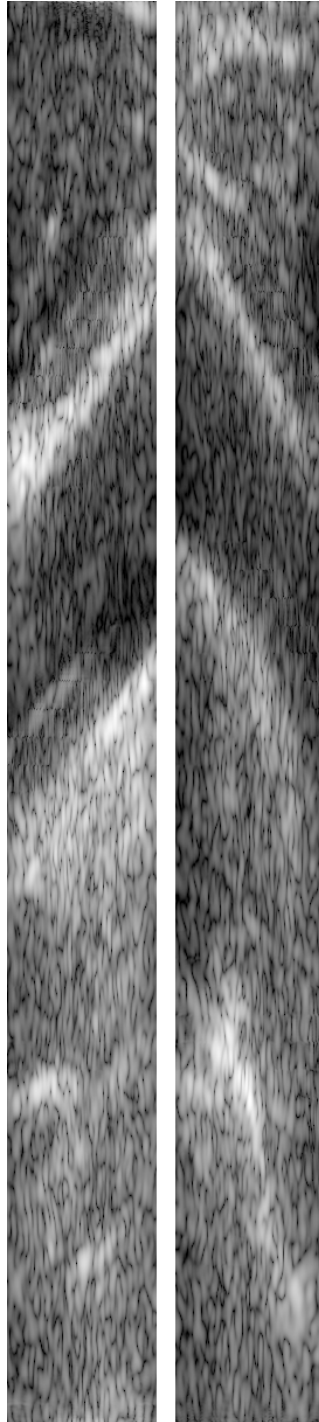


Runner figure dynamic_range_40_50_60dB.png: $B = (\text{clip}(\text{envelope_db}, -\text{DR}, 0) + \text{DR}) / \text{DR}$ for DR = 40, 50, 60 dB (the notebook's own formula). The runner confirmed that its 60 dB image is identical to the notebook's bmode.

Frame	DR	Mean brightness	Black (clipped) pixels
carotid_1	40 dB	0.210	26.44 %
carotid_1	50 dB	0.341	4.90 %
carotid_1	60 dB	0.447	0.50 %
carotid_2	40 dB	0.168	30.11 %
carotid_2	50 dB	0.303	5.74 %
carotid_2	60 dB	0.415	0.63 %

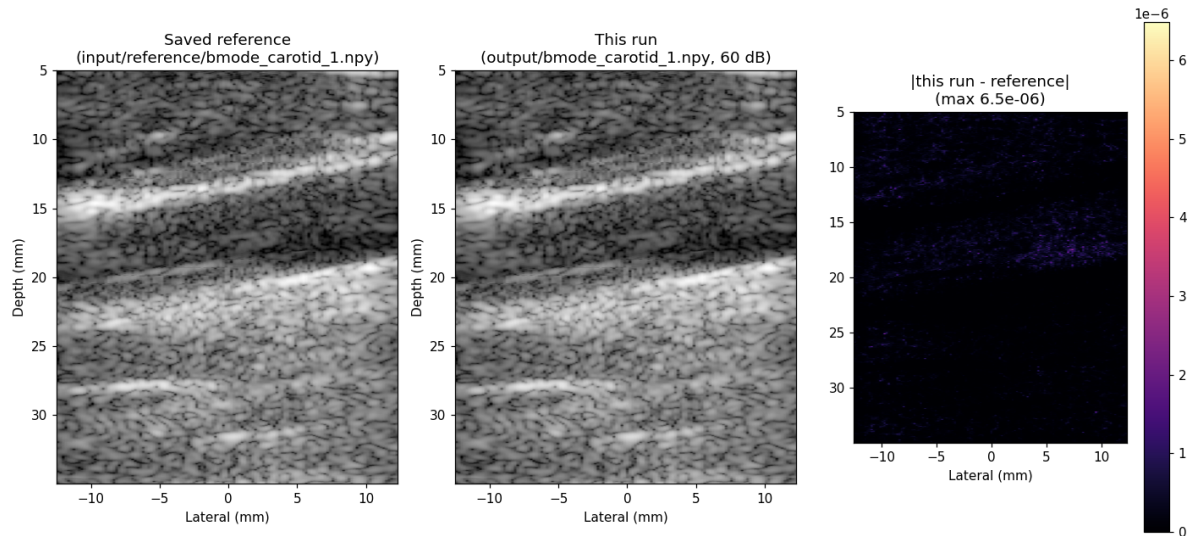
5.4 Saved files for Team 8

Cell 9 wrote **bmode_carotid_1.npy / .png / _axes.npz** and **bmode_carotid_2.npy / .png / _axes.npz** into output/: brightness (1218, 128) float32 in 0..1, the grayscale PNG and the axes (depth 5.0 to 35.0 mm, width -12.5 to 12.3 mm). The two saved PNGs (one pixel per sample, so they look tall and narrow):



bmode_carotid_1.png (left) and bmode_carotid_2.png (right) exactly as saved: 128 x 1218 pixels each, grey levels 0-255. Displayed with the mm axes (as in 5.1/5.2), the image gets its true shape.

5.5 Reproducibility check



Runner figure reproducibility_check_60dB.png: the reference image saved earlier by the notebook (input/reference/bmode_carotid_1.npy), this run's output/bmode_carotid_1.npy, and |this run - reference|.

Comparison (carotid_1, 60 dB): this run vs reference	Result
Max difference of the brightness 0..1	6.49e-06
Mean difference of the brightness 0..1	1.11e-07
8-bit grey levels round(255 B)	6 of 155904 pixels differ, by 1 level
depth_mm and width_mm axes	identical

One grey level is $1/255 = 0.0039$, so the two images are the same picture. The tiny differences are float32 rounding differences between the team's earlier notebook run (which started from the raw RF) and this run (which starts from the envelope_db in Team 6's file). The 6 pixels that change by one grey level sit exactly on a rounding boundary.

5.6 Console output

The complete console text of the run (also saved as output/console_log.txt):

python3 run_team7.py

```
=====
TEAM 7 - B-MODE IMAGE FORMATION (NITK-UsoundSim)
=====
Stage order: Team 6 Image Reconstruction (Hilbert envelope + log compression)
-> Team 7 B-mode Image Formation (THIS STAGE: dynamic range, gray levels, axes, final image)
-> Team 8 Post-Image Processing (despeckling)
Folder : Team_7_B_mode_Image_Formation
Python : 3.12.4 | numpy 1.26.4 | matplotlib 3.8.3

=====
PART 1: INPUT = output of Team 6 (Image Reconstruction), copied into input/
=====
input/reconstructed_carotid_1.npz
rf          shape (1218, 128), float32, values -162577 to 166418
envelope    shape (1218, 128), float32, values 4.99583 to 170356
envelope_norm shape (1218, 128), float32, values 2.93259e-05 to 1
envelope_db shape (1218, 128), float32, values -90.655 to 0
fs          scalar float64 = 31250000.0
f0          scalar float64 = 5000000.0
c           scalar float64 = 1540.0
pitch       scalar float64 = 0.0001953
depth_start_mm scalar float64 = 5.0
input/reconstructed_carotid_2.npz
```

```

rf          shape (1218, 128), float32, values -130418 to 139393
envelope    shape (1218, 128), float32, values 7.11421 to 143632
envelope_norm shape (1218, 128), float32, values 4.95308e-05 to 1
envelope_db  shape (1218, 128), float32, values -86.1025 to 0
fs          scalar float64 = 31250000.0
f0          scalar float64 = 5000000.0
c           scalar float64 = 1540.0
pitch       scalar float64 = 0.0001953
depth_start_mm scalar float64 = 5.0
[figure saved] output/input_envelope_db_overview.png (runner figure)

=====
PART 2.1: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_1.npz
=====
Loaded into the namespace from the input file: rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitc
↳ h, depth_start_mm
Runner sets DATA_FILE = "carotid_1_rf.npy" (only used by cell 9 to name the saved files).
Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,
cell 5 sections 3-4 (normalise, log compression).
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B_mode.ipynb code cell 2 (settings; gives dynamic_range)
--- B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, brightness 0..1)
B-mode shape: (1218, 128)
Brightness range: 0.0 to 1.0
--- B_mode.ipynb code cell 6 (depth and width axes)
Depth: 5.0 mm to 35.0 mm
Width: -12.5 mm to 12.3 mm
--- B_mode.ipynb code cell 7 (display all steps)
[figure saved] output/carotid_1_01_all_steps.png
--- B_mode.ipynb code cell 8 (final B-mode image)
[figure saved] output/carotid_1_02_final_bmode.png
--- B_mode.ipynb code cell 9 (save, run inside output/)
Saved: bmode_carotid_1.npy | bmode_carotid_1.png | bmode_carotid_1_axes.npz
Image size: (1218, 128) (depth samples x scan lines)

=====
PART 2.2: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_2.npz
=====
Loaded into the namespace from the input file: rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitc
↳ h, depth_start_mm
Runner sets DATA_FILE = "carotid_2_rf.npy" (only used by cell 9 to name the saved files).
Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,
cell 5 sections 3-4 (normalise, log compression).
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B_mode.ipynb code cell 2 (settings; gives dynamic_range)
--- B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, brightness 0..1)
B-mode shape: (1218, 128)
Brightness range: 0.0 to 1.0
--- B_mode.ipynb code cell 6 (depth and width axes)
Depth: 5.0 mm to 35.0 mm
Width: -12.5 mm to 12.3 mm
--- B_mode.ipynb code cell 7 (display all steps)
[figure saved] output/carotid_2_01_all_steps.png
--- B_mode.ipynb code cell 8 (final B-mode image)
[figure saved] output/carotid_2_02_final_bmode.png
--- B_mode.ipynb code cell 9 (save, run inside output/)
Saved: bmode_carotid_2.npy | bmode_carotid_2.png | bmode_carotid_2_axes.npz
Image size: (1218, 128) (depth samples x scan lines)

=====
PART 3: effect of the dynamic range (runner figure, notebook formula)
=====
For DR = 40, 50, 60 dB:  $B = (\text{clip}(\text{envelope\_db}, -\text{DR}, 0) + \text{DR}) / \text{DR}$  from the input envelope_db
carotid_1, DR 40 dB: mean brightness 0.210, black (clipped) pixels 26.44 %, pixels at full white 1
carotid_1, DR 50 dB: mean brightness 0.341, black (clipped) pixels 4.90 %, pixels at full white 1
carotid_1, DR 60 dB: mean brightness 0.447, black (clipped) pixels 0.50 %, pixels at full white 1
carotid_2, DR 40 dB: mean brightness 0.168, black (clipped) pixels 30.11 %, pixels at full white 1
carotid_2, DR 50 dB: mean brightness 0.303, black (clipped) pixels 5.74 %, pixels at full white 1
carotid_2, DR 60 dB: mean brightness 0.415, black (clipped) pixels 0.63 %, pixels at full white 1
[figure saved] output/dynamic_range_40_50_60dB.png (runner figure)
60 dB image here identical to the notebook's bmode for carotid_1: True

=====
PART 4: reproducibility check (this run vs the image the notebook saved earlier)
=====

```

```
=====
this run  output/bmode_carotid_1.npy          : shape (1218, 128), float32, values 0 to 1
reference input/reference/bmode_carotid_1.npy : shape (1218, 128), float32, values 0 to 1
max |this run - reference|                    = 6.49e-06
mean |this run - reference|                  = 1.11e-07
8-bit gray levels round(255 * B): 6 of 155904 pixels differ, by at most 1 level (values that sit right
↳ on a rounding boundary)
depth_mm and width_mm identical to reference : True
RESULT: PASS - same image (difference is float32 rounding, far below one gray level 1/255 = 0.0039)
[figure saved] output/reproducibility_check_60dB.png (runner figure)
=====
```

PART 5: SUMMARY

```
=====
carotid_1:
input envelope_db (1218, 128), float32, values -90.655 to 0 dB
dynamic range     60 dB -> clipped dB values -60 to 0 dB
B-mode brightness float32, values 0 to 1, mean 0.447
8-bit gray levels 0 to 255 (round(255 * B)): mean 114.1
axes              depth 5.0 to 35.0 mm, lateral -12.5 to 12.3 mm
black pixels      40 dB: 26.44 %, 50 dB: 4.90 %, 60 dB: 0.50 %
carotid_2:
input envelope_db (1218, 128), float32, values -86.1025 to 0 dB
dynamic range     60 dB -> clipped dB values -60 to 0 dB
B-mode brightness float32, values 0 to 1, mean 0.415
8-bit gray levels 0 to 255 (round(255 * B)): mean 105.9
axes              depth 5.0 to 35.0 mm, lateral -12.5 to 12.3 mm
black pixels      40 dB: 30.11 %, 50 dB: 5.74 %, 60 dB: 0.63 %
Reproducibility (carotid_1): max |this run - reference| = 6.49e-06 (PASS)
```

Timings:

Part 1 input overview	0.42 s
Part 2.1 notebook stage, carotid_1	0.92 s
Part 2.2 notebook stage, carotid_2	0.72 s
Part 3 dynamic range comparison	0.48 s
Part 4 reproducibility check	0.00 s
TOTAL	2.83 s

Files written to output/:

```
bmode_carotid_1.npy
bmode_carotid_1.png
bmode_carotid_1_axes.npz
bmode_carotid_2.npy
bmode_carotid_2.png
bmode_carotid_2_axes.npz
carotid_1_01_all_steps.png
carotid_1_02_final_bmode.png
carotid_2_01_all_steps.png
carotid_2_02_final_bmode.png
dynamic_range_40_50_60dB.png
input_envelope_db_overview.png
reproducibility_check_60dB.png
console_log.txt
console_screenshot_1.png, _2.png, ... (pictures of this console text, 75 lines each)
```

Done.

```

Terminal - python3 run_team7.py (part 1 of 3)

=====
TEAM 7 - B-MODE IMAGE FORMATION (NITK-UsoundSim)
=====
Stage order: Team 6 Image Reconstruction (Hilbert envelope + log compression)
-> Team 7 B-mode Image Formation (THIS STAGE: dynamic range, gray levels, axes, final image)
-> Team 8 Post-Image Processing (despeckling)
Folder : Team 7 B-mode Image Formation
Python : 3.12.4 | numpy 1.26.4 | matplotlib 3.8.3

=====
PART 1: INPUT = output of Team 6 (Image Reconstruction), copied into input/
=====
input/reconstructed_carotid_1.npz
rf          shape (1218, 128), float32, values -162577 to 166418
envelope    shape (1218, 128), float32, values 4.99583 to 170356
envelope_norm shape (1218, 128), float32, values 2.93259e-05 to 1
envelope_db  shape (1218, 128), float32, values -90.655 to 0
fs          scalar float64 = 31250000.0
f0          scalar float64 = 5000000.0
c           scalar float64 = 1540.0
pitch       scalar float64 = 0.0001953
depth_start_mm scalar float64 = 5.0
input/reconstructed_carotid_2.npz
rf          shape (1218, 128), float32, values -130418 to 139393
envelope    shape (1218, 128), float32, values 7.11421 to 143632
envelope_norm shape (1218, 128), float32, values 4.95308e-05 to 1
envelope_db  shape (1218, 128), float32, values -86.1025 to 0
fs          scalar float64 = 31250000.0
f0          scalar float64 = 5000000.0
c           scalar float64 = 1540.0
pitch       scalar float64 = 0.0001953
depth_start_mm scalar float64 = 5.0
[figure saved] output/input_envelope_db_overview.png (runner figure)

=====
PART 2.1: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_1.npz
=====
Loaded into the namespace from the input file: rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitch, depth_start_mm
Runner sets DATA FILE = "carotid_1_rf.npy" (only used by cell 9 to name the saved files).
Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,
cell 5 sections 3-4 (normalise, log compression).
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B_mode.ipynb code cell 2 (settings; gives dynamic range)
--- B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, brightness 0..1)
B-mode shape: (1218, 128)
Brightness range: 0.0 to 1.0
--- B_mode.ipynb code cell 6 (depth and width axes)
Depth: 5.0 mm to 35.0 mm
Width: 12.5 mm to 12.3 mm
--- B_mode.ipynb code cell 7 (display all steps)
[figure saved] output/carotid_1_01_all_steps.png
--- B_mode.ipynb code cell 8 (final B-mode image)
[figure saved] output/carotid_1_02_final_bmode.png
--- B_mode.ipynb code cell 9 (save, run inside output/)
Saved: bmode_carotid_1.npy | bmode_carotid_1.png | bmode_carotid_1_axes.npz
Image size: (1218, 128) (depth samples x scan lines)

=====
PART 2.2: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_2.npz
=====
Loaded into the namespace from the input file: rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitch, depth_start_mm
Runner sets DATA FILE = "carotid_2_rf.npy" (only used by cell 9 to name the saved files).
Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,
cell 5 sections 3-4 (normalise, log compression).
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B_mode.ipynb code cell 2 (settings; gives dynamic range)
--- B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, brightness 0..1)
B-mode shape: (1218, 128)
Brightness range: 0.0 to 1.0
--- B_mode.ipynb code cell 6 (depth and width axes)
Depth: 5.0 mm to 35.0 mm
Width: 12.5 mm to 12.3 mm
--- B_mode.ipynb code cell 7 (display all steps)
[figure saved] output/carotid_2_01_all_steps.png

```

output/console_screenshot_1.png: picture of the first part of the console output (parts 2 and 3 are in the same folder).

6 How to run

Step by step, for macOS, Linux and Windows

1. Install Python 3 (3.9 or newer) from <https://www.python.org>. On Windows tick 'Add Python to PATH'.
2. Open a terminal: macOS Terminal, Linux terminal, or Windows Command Prompt / PowerShell.
3. Go into the folder: **cd path/to/Team_7_B_mode_Image_Formation** (Windows: `cd path\to\Team_7_B_mode_Image_Formation`).
4. Install the packages once: **python3 -m pip install -r requirements.txt** (Windows: `python -m pip install -r requirements.txt`).
5. Run: **python3 run_team7.py** (Windows: **python run_team7.py**).
6. Look at the results in **output/** (PNG figures, `bmode_*.npy/.png/_axes.npz`, `console_log.txt`, console pictures).

Expected runtime: about 3 seconds (the script measured 2.83 s on a Mac laptop; a first run can take a few seconds longer while libraries load). No windows open: all figures are saved as files.

Jupyter: opening `B_mode.ipynb` directly runs the whole chain including Team 6's part and needs the raw RF files `carotid_1_rf.npy` / `carotid_2_rf.npy` next to it (and `scipy` for the Hilbert transform); they are Team 6's input and not in this folder.

Troubleshooting

Problem	What to do
No module named numpy / matplotlib	Run the pip install step again with the same python3 / python command
python3 not found (Windows)	Use python instead of python3
FileNotFoundError: ...reconstructed_carotid_1.npz	Run from inside the folder; keep input/ next to run_team7.py
No picture windows	Normal: figures are saved in output/
Numbers differ by about 1e-6	Different numpy versions round float32 slightly differently

7 Check yourself

Questions with short answers

What exactly does Team 7 receive from Team 6, and what is 0 dB?

The log-compressed envelope $\text{envelope_db} = 20 \cdot \log_{10}(E/\max E + 1e-12)$ for every pixel, not yet clipped. 0 dB is the strongest echo of the frame; all other values are negative (down to -90.655 dB for carotid_1).

What does the dynamic range do?

It decides how many dB below the peak are displayed. Values below -DR are clipped to -DR and become black; the range -DR..0 dB is spread over black..white.

What happens to the image when DR goes from 60 dB to 40 dB?

Higher contrast and a darker image: more weak echoes are clipped. In this run the black pixels of carotid_1 rise from 0.50 % to 26.44 % and the mean brightness falls from 0.447 to 0.210.

With DR = 60 dB, what brightness and grey level does a pixel at -45 dB get?

$B = (-45 + 60)/60 = 0.25$, grey level $\text{round}(255 \cdot 0.25) = 64$ (dark grey).

How is depth computed, and why the factor 2?

$z = z_0 + c \cdot i/(2 \cdot f_s)$. i/f_s is the time since the start; the sound travels down and back, so the reflector is at half the distance. Here 5.0 to 35.0 mm.

Why does the saved PNG look tall and thin, but the figure does not?

The array has 1218 rows and 128 columns (one pixel per sample). The figure uses extent in mm, so 30 mm of depth and 24.8 mm of width are drawn at their true size.